WebRelate: Integrating Web Data with Spreadsheets using Examples

ANONYMOUS AUTHOR(S)

Data integration between web sources and relational data is a key challenge faced by data scientists and spreadsheet users. There are two main challenges in programmatically joining web data with relational data. First, most websites do not expose a direct interface to obtain tabular data, so the user needs to formulate a logic to get to different webpages for each input row in the relational table. Second, after reaching the desired webpage, the user needs to write complex scripts to extract the relevant data, which is often conditioned on the input data. Since many data scientists and end-users come from diverse backgrounds, writing such complex regular-expression based logical scripts to perform data integration tasks is unfortunately often beyond their programming expertise.

We present WebRelate, a system that allows users to join semi-structured web data with relational data in spreadsheets using input-output examples. WebRelate decomposes the web data integration task into two sub-tasks. The first sub-task generates the URLs for the webpages containing the desired data for all rows in the relational table. WebRelate achieves this by learning a string transformation program using a few example URLs. The second sub-task uses examples of desired data to be extracted from the corresponding webpages and learns a program to extract the data for the other rows. We design expressive domain-specific languages for URL generation and web data extraction, and present efficient synthesis algorithms for learning programs in these DSLs from few input-output examples. We evaluate WebRelate on 88 real-world data integration tasks taken from online help forums and other anonymous sources, and show that WebRelate can learn the programs to perform desired web data integration tasks within few seconds using only 1 example for the majority of the tasks.

ACM Reference format:
Anonymous Author(s). 2017. WebRelate: Integrating Web Data with Spreadsheets using Examples. *Proc. ACM Program. Lang.* 1, 1, Article 1 (January 2017), 27 pages.
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Data integration is a key challenge faced by many data scientists and spreadsheet end-users. Despite several recent advances in techniques for making it easier for users to perform data analysis [Kandel et al. 2011], the inferences obtained from the analyses is only as good as the information diversity of the data. Therefore, more and more users are enriching their internal data in spreadsheets with the rich data available on the web. However, the web data sources (websites) are typically semi-structured and in a format different from the original spreadsheet data format, and hence, performing such integration tasks requires writing complex regular-expression based data transformation and scraping scripts. Unfortunately, a large fraction of end-users come from diverse backgrounds and writing such scripts is beyond their programming expertise [Gualtieri 2009]. Even for experienced programmers and data scientists, writing such data integration scripts can be difficult and time consuming.

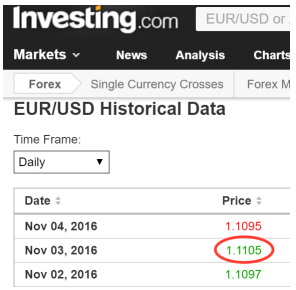
Example 1.1. To make the problem concrete, consider the integration task shown in Fig. 1. A user had a spreadsheet consisting of pairs of currencies and the dates of transaction (shown in

A note.
2017. 2475-1421/2017/1-ART1 $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Fig. 1(a)), and wanted to get the exchange rates from the web. Since there were thousands of rows in the spreadsheet, manually performing this task was prohibitively expensive for the user.

	Cur 1	Cur 2	Date	Exchange Rate
1	EUR	USD	03, November, 16	
2	USD	INR	01, November, 16	
3	AUD	CAD	07, October, 16	

(a)



(b)

Fig. 1. (a) A spreadsheet containing pairs of currency symbols with the currency exchange rates. (b) Webpage for EUR to USD exchange rate.

Previous works have explored two different strategies for automating such data integration tasks. In DataXFormer [Abedjan et al. 2015], a user provides a few end-to-end input-output examples (such as $row1 \rightarrow 1.1105$ in the above example), and the system uses a fully automatic approach by searching through a huge database of web forms and web tables to find a transform that is consistent with the examples. However, many websites do not expose web forms and do not have the data in a single web table. On the other end are programming by demonstration (PBD) systems such as WebCombine [Chasins et al. 2015] and Vegemite [Lin et al. 2009] that rely on users to demonstrate how to navigate and retrieve the desired data for a few example rows. Although the PBD systems can handle a broader range of webpages compared to DataXFormer, they tend to put an additional burden on users to perform exact demonstrations to get to the webpage, which has been shown to be problematic for users [Lau 2009].

In this paper, we present an intermediate approach where a user provides a few examples of URLs of webpages that contain the data as well as examples of the desired data from these webpages and the system automatically learns how to perform the integration task for the other rows in the spreadsheet. For instance, in the above task, the example URL for the first row is http://www.investing.com/currencies/eur-usd-historical-data and the corresponding webpage is shown in Fig. 1(b) where the user highlights the desired data. There are three main challenges for a system to learn a program to automate this integration task.

First, the system needs to learn a program to generate the desired URLs for the other input rows. In the above example, the intended program for learning URLs needs to select the appropriate columns from the spreadsheet, perform casing transformations, and then concatenate them with appropriate constant strings. Moreover, many URL generation programs require using string transformations on input data based on complex regular expressions.

The second challenge is to learn an extraction logic to retrieve the relevant data, which depends on the underlying DOM structure of the webpage. Additionally, for many integration scenarios, the data that needs to be extracted from the web is conditioned on the data in the table. For instance, in the above example, the currency exchange rate from the web page should be extracted based on the `Date` column in the table. There are efficient systems in the literature for wrapper induction [Anton 2005; Dalvi et al. 2009; Kushmerick 1997; Muslea et al. 1998] that can learn robust and generalizable extraction programs from a few labeled examples, but, to the best of our knowledge, there is no previous work that can learn data extraction programs that are conditioned on the input.

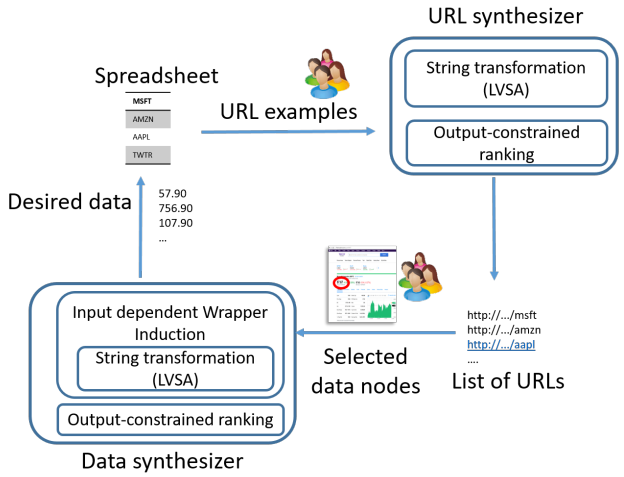


Fig. 2. An overview of the workflow of the WebRelate system. A user starts with providing a few examples for URL strings corresponding to the desired webpages of the first few spreadsheet rows. The URL synthesizer then learns a program consistent with the URL examples, which is executed to generate the desired URLs for the remaining spreadsheet rows. The user then opens the URLs for the first few spreadsheet rows in an adjoining pane (one at a time), and highlights the data items that need to be extracted from the webpage. The Data synthesizer then learns a data extraction program consistent with the examples to extract the desired data for the remaining spreadsheet rows.

The third challenge is that the common data key to join the two sources (spreadsheet and web data) might be in different formats, which requires writing additional logic to transform the data before performing the join. For instance, in the example above, the format of the date in the web page (`Nov 03, 2016`) is different from the format in the spreadsheet (`03, November, 16`).

To address the above challenges, we present WebRelate, a system that uses input-output examples to learn a program to perform the web data integration task. The high-level overview of the system is shown in Fig. 2. It breaks down the integration task into two sub-tasks. The first sub-task uses the *URL synthesizier* module to learn a program to generate the URLs of the webpages that contain the desired data from few example URLs. The second sub-task uses the *Data synthesizer* module to learn a data extraction program to retrieve the relevant data from these URLs given a set of example data selections on the webpages.

We design an expressive domain-specific language (DSL) for the URL generation programs. The DSL is built on top of regular expression based substring operations introduced in FlashFill [Gulwani 2011]. We then present a synthesis algorithm based on *layered version space algebra* (LVSA) to efficiently learn URL programs in the DSL from few input-output examples. We also show that this algorithm is significantly better than existing VSA based techniques. Learning URLs as a string transformation program raises an additional challenge since some URLs may contain additional strings such as unique identifiers that are not in the spreadsheet table and are not constants (Ex 2.2 illustrates this challenge). To handle such scenarios, the language for URLs supports learning *filter* programs that produce regular expressions for URLs as opposed to concrete strings. Then, WebRelate leverages a search engine to get a list of relevant URLs for every input and selects the one that matches the regular expression.

Similar to URL learning, WebRelate uses examples to learn data extraction programs. A user can select a URL to be loaded in a neighboring frame, and then highlight the desired data element to be extracted. WebRelate records the DOM locations of all such example data and learns a program to perform the desired extraction task for the other rows in the table. We present a new technique for *input dependent wrapper induction* that involves designing an expressive DSL built on top of XPath constructs and regular expression based substring operations to express input-dependent data extraction tasks. We, then, present a synthesis algorithm that uses *predicates graphs* to succinctly represent a large number of DSL expressions that are consistent with a given set of I/O examples. For the above example, our system learns a program that first transforms the date to the required format and then extracts the conversion rate element whose left sibling contains the transformed date value.

Both the URL synthesis and the data extraction synthesis problems fall into a class of PBE problems that we term as *Output-constrained* Programming by Example (O-PBE) problems (see § 3) that, in addition to input-output examples, have constraints on the possible outputs for unseen inputs. For URL problems, the constraint is that the output should be a valid URL or the output should be from the list of possible URLs obtained using the search engine for that particular input. For data extraction problems, the constraint is that the output program for every input should result in a non empty node in the corresponding HTML document. WebRelate leverages these additional constraints to devise an *output-constrained ranking* scheme that selects the best program from the large set of DSL programs that are consistent with the given examples.

This paper makes the following key contributions:

- We present WebRelate, a system to join web data with relational data, which divides the data integration task into URL learning and web data extraction tasks.
- We design a DSL for URL generation using string transformations and an algorithm based on *layered version space algebra* to learn URLs from a few examples (§ 4).
- We design a DSL on top of XPath constructs that allows input-dependent data extractions. We present a synthesis algorithm based on a *predicates graph* data structure to learn extraction programs from examples (§ 5).
- We evaluate WebRelate on 88 real-world data integration tasks. It takes on average less than 1.2 examples and 0.15 seconds each to learn the URLs, whereas it takes less than 5 seconds and 1 example to learn data extraction programs for 93% of tasks (§ 6).

2 MOTIVATING EXAMPLES

In this section, we present a few additional motivating scenarios for web data integration tasks of varying complexity and illustrate how WebRelate can be used to automate these tasks using few input-output examples.

Example 2.1. **[Stock prices]** A user wanted to retrieve the stock prices for hundreds of companies (`Company` column) as shown in Fig. 3. Since the spreadsheet contained hundreds of company symbols, manually entering the values was time consuming. Unfortunately, the user did not have programming expertise to write a web scraping script to automate the web extraction, and posted the request on a help forum.

In order to perform this integration task in WebRelate, a user can provide an example URL such as https://finance.yahoo.com/q?s=msft that has the desired stock price for the first row in the spreadsheet (Fig. 3(a)). This web-page then gets loaded and is displayed to the user. The user can then highlight the required data from the web-page (Fig. 3(b)). WebRelate learns the desired program to perform this task in two steps. First, it learns a program to generate the URLs for remaining rows by learning a string transformation program that combines the constant string

	Company	Stock price	URL
1	MSFT	59.87	https://finance.yahoo.com/q?s=msft
2	AMZN	**775.88**	**https://finance.yahoo.com/q?s=amzn**
3	AAPL	**113.69**	**https://finance.yahoo.com/q?s=aapl**
4	TWTR	**17.66**	**https://finance.yahoo.com/q?s=twtr**
5	T	**36.51**	**https://finance.yahoo.com/q?s=t**
6	S	**6.31**	**https://finance.yahoo.com/q?s=s**

(a)

(b)

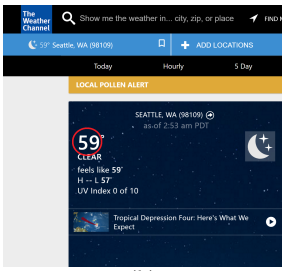
Fig. 3. Joining stock prices from web with company symbols using WebRelate. Given one example URL and data extraction from the webpage for the first row, the system learns a program to automatically generate the corresponding URLs and extracted stock prices for the other rows (shown in bold).

"https://finance.yahoo.com/q?s=" with the company symbol in the input. Second, it learns a web data extraction program to extract the desired data from these web-pages.

Example 2.2. **[Weather]**. A user had a list of addresses in a spreadsheet and wanted to get the weather information at each location as shown in Fig. 4. The provided example URL is https://weather.com/weather/today/l/Seattle+WA+98109:4:US#!.

	Address	Weather
1	742 17th Street NE,Seattle,WA	59
2	732 Memorial Drive,Cambridge,MA	**43**
3	Apt 12, 112 NE Main St.,Boston,MA	**42**

(a)



(b)

Fig. 4. Joining weather data from web with addresses. Given one example row, the system automatically extracts the weather for other row entries (shown in bold).

There are two challenges in learning the URL generation program for this example. First, the addresses contain more information than just the city and state names such as street name and house number. Therefore, the URL generator program first needs to learn regular expressions to extract the city-name and state from the address and then concatenate them appropriately with constant strings to get the desired URL. The second challenge is that the URL contains zip code that is not present in the input, meaning that there is no simple program that concatenates constants and sub-strings to learn the URLs for the remaining inputs. For supporting such cases, the DSL for URL learning also supports *filter* programs that use regular expressions to denote unknown parts of the URL. A possible *filter* program for this example is https://weather.com/weather/today/l/{**Extracted city name**}+{**Extracted state name**}+{**AnyStr**}:4:US#!, where **AnyStr** can match any non-empty string. Then, for every other row in the table, WebRelate leverages a search engine to get a

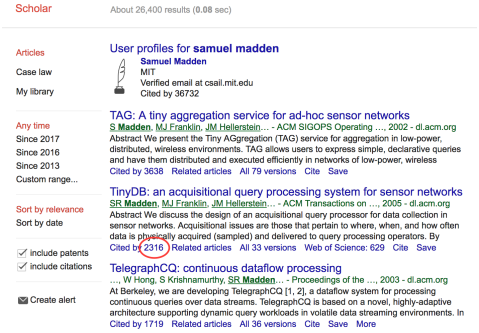
list of possible URLs for that row and selects the top URL that matches the filter program. By default, we use the words in input row as the search query term and set the target URL domain to be the domain of the given example URLs. WebRelate also allows users to provide search query terms (such as "Seattle weather") as additional search query examples and it learns the remaining search queries for other inputs using a string transformation program. Using the search query and a filter program, WebRelate learns the following URL for the second row: https://weather.com/weather/today/l/Cambridge+MA+02139:4:US#!.

Example 2.3. **[Citations]** A user had a table containing author names and titles of research papers, and wanted to extract the number of citations for each article using Google Scholar (as shown in Fig. 5).

In this case, the example URL for Samuel Madden's Google Scholar page is https://scholar.google.com/scholar?q=samuel+madden and the corresponding web page is shown in Fig. 5(b). The URL can be learned as a string transformation program over the `Author` column. The more challenging part of this integration task is that the data in the web page is in a semi-structured format and the required data (# citations) should be extracted based on the `Article` column in the input. Our data extraction DSL is expressive enough to learn a program that captures this complex dependency. This program generates the entire string "Cited by 2316" and WebRelate learns another transformation program on top of this to extract only the number of citations "2316" from the results.

	Author	Article	# citations
1	Samuel Madden	TinyDB: an acquisitional ...	2316
2	HV Jagadish	Structural joins: A primitive ...	**1157**
3	Mike Stonebraker	C-store: a column-oriented ...	**1119**

(a)



(b)

Fig. 5. (a) Integrating a spreadsheet containing authors and article titles with the number of citations obtained from Google Scholar. (b) The google scholar page for the first example

3 PROBLEM FORMULATION

We first define the abstract *Output-constrained* Programming By Example (O-PBE) problem, which we then instantiate for both the URL synthesizer and the data extraction synthesizer. Let $E = \{(i_1, o_1), \cdots, (i_m, o_m)\}$ denote the list of m input-output examples provided by the user and $U = \{i_{m+1}, \cdots, i_{m+n}\}$ denote the list of n unseen inputs (inputs with unknown outputs). We use L to denote the DSL that describes the space of possible programs. The traditional PBE problem is to find a program P in the DSL that satisfies the given input-output examples i.e. $\exists\, P \in L.\ \forall\, k \leq m.\ P(i_k) = o_k$. But unlike the traditional PBE problem, in this approach, we also take into account the existence of a generator function G that can generate a list of possible outputs for a given input. For instance, in the URL learning scenario, the generator G can be a search engine that lists the top URLs using a search query term based on the input. On the other hand, for the data extraction learning scenario, the generator G can be the web document corresponding to each input such that any node in

$$
\begin{array}{rcl}
\text{URL String } u & := & \texttt{Filter}(\phi) \mid e \\
\text{String } e & := & \texttt{Concat}(b_1, \cdots, b_n) \\
\text{Predicate } \phi & := & \texttt{Concat}(f_1, \cdots, f_n) \\
\text{Atomic expr } f & := & r \mid b \\
\text{Regex expr } r & := & \texttt{AnyStr} \\
\text{Base expr } b & := & \texttt{ConstStr}(s) \mid \texttt{SubStr}(i, p_l, p_r, c) \mid \texttt{Replace}(i, p_l, p_r, c, s_1, s_2) \\
\text{Position } p & := & (\tau, k, \texttt{Dir}) \mid \texttt{ConstPos}(k) \\
\text{Direction } \texttt{Dir} & := & \texttt{Start} \mid \texttt{End} \\
\text{Token } \tau & := & s \mid T \\
\text{Regex Token } T & := & \texttt{CAPS} \mid \texttt{ProperCase} \mid \texttt{lowercase} \mid \texttt{Digits} \mid \texttt{Alphabets} \mid \texttt{AlphaNum} \\
\text{Case } c & := & \texttt{lower} \mid \texttt{upper} \mid \texttt{prop} \mid \texttt{iden}
\end{array}
$$

Fig. 6. The syntax of the DSL L_u for regular expression based URL learning. Here, s is a string, and k is an integer.

the web document becomes a candidate output. The generator G constrains the space of outputs that can be produced by the learned programs when run on the corresponding inputs. With this additional information, we can have more expressive higher-order expressions in the language L that takes as input a list of possible outputs (in addition to the input) and chooses one among them i.e. $P(i, G(i)) = o \; s.t. \; o \in G(i)$. Now, the O-PBE problem can be defined as follows:

$$\exists \, P \in L. \; \forall \, k \leq m. \; P(i_k, G(i_k)) = o_k \;\; \wedge \;\; \forall \, k \leq n \; P(i_{m+k}, G(i_{m+k})) \in G(i_{m+k})$$

At a high level, to solve this synthesis problem, WebRelate first learns the set of all programs in the language L that satisfies the given input-output examples E. This set is represented succinctly using special data structures. Then, WebRelate uses an *output-constrained ranking scheme* to eliminate programs that are inconsistent with the unseen inputs U, i.e. violate the output-constraint provided by G. This allows WebRelate to efficiently learn the desired programs using very few examples.

We now describe the two instantiations of the abstract O-PBE problem for the URL and data extraction synthesizer in more detail.

4 URL LEARNING

We first present the domain-specific language for the URL generation programs and then present a synthesis algorithm based on *layered version space algebra* to efficiently learn programs from few input-output examples.

4.1 URL Generation Language L_u

Syntax. The syntax of the DSL for URL generation L_u is shown in Fig. 6. The DSL is built on top of regular expression based substring constructs introduced in FlashFill [Gulwani 2011; Gulwani et al. 2012] and the key differences from the FlashFill DSL are highlighted. The top-level URL string expression u can either be a filter expression $\texttt{Filter}(\phi)$ or a string expression e. The Filter expression takes as argument a predicate ϕ. Both e and ϕ are denoted using concatenate expressions, but the difference between the two Concat expressions is that while e is constructed using concatenation of only constant strings, regular expression based substring expressions,

$$
\begin{aligned}
[\![\texttt{Filter}(\phi)]\!]_v &= u \text{ s.t. } u \in G_u(v) \text{ and } [\![\phi]\!]_v \models u \\
[\![\texttt{Concat}(b_1, \cdots, b_n)]\!]_v &= \texttt{Concat}([\![b_1]\!]_v, \cdots, [\![b_n]\!]_v) \\
[\![\texttt{Concat}(f_1, \cdots, f_n)]\!]_v &= \texttt{Concat}([\![f_1]\!]_v, \cdots, [\![f_n]\!]_v) \\
[\![\texttt{AnyStr}]\!]_v &= \Sigma^+ \\
[\![\texttt{ConstStr}(s)]\!]_v &= s \\
[\![\texttt{SubStr}(i, p_l, p_r, c)]\!]_v &= \texttt{ToCase}(v_i[[\![p_l]\!]_v..[\![p_r]\!]_v], c) \\
[\![\texttt{Replace}(i, p_l, p_r, c, s_1, s_2)]\!]_v &= \texttt{ToCase}(v_i[[\![p_l]\!]_v..[\![p_r]\!]_v], c)[s_1 \leftarrow s_2] \\
[\![\texttt{ConstPos}(k)]\!]_v &= k > 0 ?\ k : \texttt{len}(s) + k \\
[\![(\tau, k, \texttt{Start})]\!]_v &= \text{Start of } k^{\text{th}} \text{match of } \tau \text{ in v} \\
[\![(\tau, k, \texttt{End})]\!]_v &= \text{End of } k^{\text{th}} \text{match of } \tau \text{ in v}
\end{aligned}
$$

Fig. 7. The semantics of the DSL L_u. G_u is a URL list generator that generates a ranked list of URLs for the input v by using a search engine.

and replace expressions, ϕ can additionally contain AnyStr expressions. The base atomic string expression b can either be a constant string, a substring expression that takes an index, two position expressions and a case expression, or a replace expression that takes two string arguments in addition to the arguments to substring expression. A position expression can either be a constant position index k or a regular expression based position $(\tau, k, \texttt{Dir})$ that denotes the Start or End of k^{th} match of token τ in the input string.

Semantics. The semantics of the DSL for L_u is shown in Fig. 7. We use the notation $[\![x]\!]_v$ to represent the semantics of x when evaluated on an input v (a list of strings from the table row). The semantics of a filter expression $\texttt{Filter}(\phi)$ is to use a URL list generator G_u to obtain a ranked list of URLs for the input v, and select the first URL that matches the regular expression generated by the evaluation of the predicate ϕ. The default implementation for G_u runs a search engine on the input v and returns the URLs of the top results. However, a user can also provide a few search query examples, which WebRelate uses to learn a string expression (e) to query the search engine for the remaining rows. The semantics of a string expression e and a predicate expression ϕ is to first evaluate each individual atomic expression in the arguments and then return the concatenation of resulting atomic strings. The semantics of AnyStr expression is Σ^+ that can match any non-empty string. The semantics of a substring expression is to first evaluate the position expressions to obtain left and right indices and then return the corresponding substring for the ith string in v. The semantics of a replace expression $\texttt{Replace}(i, p_l, p_r, c, s_1, s_2)$ is to first get the substring corresponding to the position expressions and then replace all occurrences of string s_1 with the string s_2 in the substring. We allow strings s_1 and s_2 to take values from a finite set of delimiter strings such as " ", "-", "_", and "#".

Examples. A program in L_u to perform the URL learning task in Ex 1.1 is: Concat(ConstStr("http://www.investing.com/currencies/"), Substr(0, ConstPos(0), ConstPos(-1), lower), ConstStr("-"), Substr(1, ConstPos(0), ConstPos(-1), lower), ConstStr("-historical-data")). The program concatenates a constant string, the lowercase transformation of the first input string (-1 index in ConstPos denotes the last string index), the constant hyphen , the lowercase transformation of the second input string, and finally another constant string.

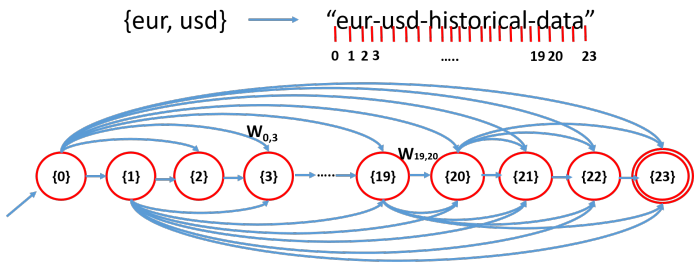


Fig. 8. A sample DAG using VSA to represent the set of transformations consistent with the example. For example, here, $W_{0,3}$ is a list of three different expressions {ConstStr(eur), SubStr(0, ConstPos(0), ConstPos(−1), lower), AnyStr};$W_{19,20}$ = {ConstStr(d), SubStr(1, ConstPos(2), ConstPos(3), lower), AnyStr}.

A DSL program for the URL learning task in Ex 2.2 is: Filter(ϕ), where $\phi \equiv$ Concat(b_1, b_2, ConstStr("+"), b_3,ConstStr("+"), AnyStr, ConstStr(":4:US#!")), $b_2 \equiv$SubStr((",",-2,End),(",",-1,Start),iden), $b_1 \equiv$ ConstStr("https://weather.com/weather/today/l/"), and $b_3 \equiv$ SubStr((",",-1,End),ConstPos(-1),iden).

4.2 Synthesis Algorithm

We now present the synthesis algorithm to learn a URL program the solves the O-PBE problem. Here, we use $E = \{(v_1, o_1), (v_2, o_2), \cdots, (v_m, o_m)\}$ to denote the input-output examples where each example is a pair of an input from the spreadsheet v and the desired URL o.

4.2.1 Background: Version Space Algebra. We first present a brief background on version space algebra (VSA) [Gulwani et al. 2012], a technique used by existing string transformation synthesizers such as FlashFill. The key idea of VSA is to use a directed acyclic graph (DAG) like structure to succinctly represent an exponential number of candidate programs using polynomial space. For the string transformation scenario, a DAG D is defined as a tuple (V, v_s, v_t, E, W) where V is the set of vertices, E is the set of edges, v_s is the starting vertex and v_t is the target vertex. Each edge in the DAG represents a set of atomic expressions (the map W captures this relation) whereas a path in the DAG represents the concatenation of all the edge expressions. Given a synthesis problem, a DAG is constructed in such a way that any path in the DAG from v_s to v_t is a valid program in L_u that satisfies the examples. This is achieved by iteratively constructing a DAG for each example and performing an automata-like-intersection on these individual DAGs to get the final DAG. For example, a sample DAG is shown in Fig. 8. The nodes in this DAG (for each I/O example) corresponds to the indices of the output string. An edge from a node i to a node j in the DAG represents the set of expressions that can generate the substring between the indices i and j of the output example string when executed on the input data.

4.2.2 Layered Version Space Algebra. It is challenging to use VSA techniques for learning URLs because URL strings are long, and the run time and the DAG size of existing algorithms explode with the length of the output To overcome this issue, we introduce a synthesis algorithm based on layered version space algebra for efficiently searching the consistent programs. The key idea is to perform search over increasingly expressive sub-languages $L_1 \subseteq L_2 \cdots \subseteq L_k$, where L_k corresponds to the complete language L_u. The intuition for selecting the sub-languages is that the earlier languages capture the portion of the search space that is highly probable and at the same time, it is easier to search for a program in these sub-languages. For example, it is unlikely to

concatenate a constant with a sub-string within a word in the URL. For instance, consider the URL in Ex 1.1. In this case, given the first I/O example, programs that derive the character `d` in `data` from the last character in the input (`USD`) are not desirable because they do not generalize to other inputs.

The general synthesis algorithm `GenProg` for learning URL string transformations in a sub-language L_i is shown in Fig. 11. This algorithm is similar to the VSA based algorithm, but the key difference is that instead of learning all candidate programs (which can be expensive), the algorithm only learns the programs that are in the sub-language L_i. The `GenProg` algorithm takes as input the input-output examples $\{v_i, o_i\}_i$, and three Boolean functions $\lambda_s, \lambda_c, \lambda_a$ that parameterize the search space. The output is a program that is consistent with the examples or $\perp$ if no such program exists. The algorithm first uses the `GenDag` procedure to learn a DAG consisting of all consistent programs (in a given search space) for the first example. It then iterates over other examples and intersects the corresponding DAGs to compute a DAG representing programs consistent with all examples. Finally, it returns the top-ranked path in the DAG. If this path does not contain any `AnyStr`, then the output is an e expression that concatenates the individual path expressions. Otherwise, the output is a *Filter* expression.

The `GenDag` algorithm is shown in Fig. 11, where the space of programs is constrained by the parameter Boolean functions λ_s, λ_c, and λ_a. Each function λ : `int` $\rightarrow$ `int` $\rightarrow$ `string` $\rightarrow$ `bool` takes two integer indices and a string as input and returns a Boolean value denoting whether certain atomic expressions are allowed to be added to the DAG. The algorithm first creates `len`(o) + 1 nodes, and then adds an edge between each pair of nodes $< i, j >$ such that $0 \leq i < j \leq$ `len`(o). For each edge $< i, j >$, the algorithm learns all atomic expressions that can generate the substring $o[i..j]$. For learning `SubStr` and `Replace` expressions, the algorithm enumerates different argument parameters for positions, cases, and delimiter strings, whereas the `ConstStr` and `AnyStr` expressions are always available to be added. The addition of `SubStr` and `Replace` atomic expressions to the DAG are guarded by the Boolean function λ_s whereas the addition of `ConstStr` and `AnyStr` atomic expressions are guarded by the Boolean functions λ_c and λ_a, respectively.

Fig. 12 shows how the different layers are instantiated for learning URL expressions. For the first layer, the algorithm only searches for string expressions where each word in the output is either a substring or a constant or an `AnyStr`. The `onlyWords` (`oW`) function is defined as:

$$\texttt{oW} = (i, j, o) \texttt{ => } \neg\texttt{isAlpha}(o[i-1]) \wedge \neg\texttt{isAlpha}(o[j+1]) \wedge \forall\, k : i \leq k \leq j \texttt{ isAlpha}(o[k])$$

The second layer allows for multiple words in the output string to be learned as a substring. The function `multipleWords` (`mW`) is defined as:

$$\texttt{mW} = (i, j, o) \texttt{ => } \neg\texttt{isAlpha}(o[i-1]) \ \wedge \ \neg\texttt{isAlpha}(o[j+1])$$

The third layer, in addition, allows for words in the output string to be a concatenation of multiple substrings, but not a concatenation of substrings with constant strings (or `AnyStr`). The function `insideWords` (`iW`) is defined as:

$$\texttt{iW} = (i, j, o) \texttt{ => } \forall\, k : i \leq k \leq j \texttt{ isAlpha}(o[k])$$

The final layer allows arbitrary compositions of constants, `AnyStr` and substring expressions by setting the functions λ_s, λ_c and λ_a to always return `True` (`T`).

Example 4.1. Consider the currency exchange example where the example URL for the input {`EUR`, `USD`, `03, November 16`} is "http://www.investing.com/currencies/eur-usd-historical-data". The first layer of the search will create a DAG as shown in Fig. 9. We can observe that the DAG eliminates most of the unnecessary programs such as those that consider the `d` in `data` to come from the `D` in `USD` and the DAG is much smaller (and sparser) compared to the DAG in Fig. 8.

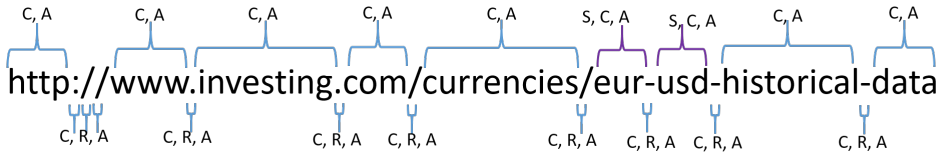


Fig. 9. DAG for Ex 1.1 constructed using layer 1 where S: SubStr, C: ConstStr, R: Replace, and A: AnyStr.

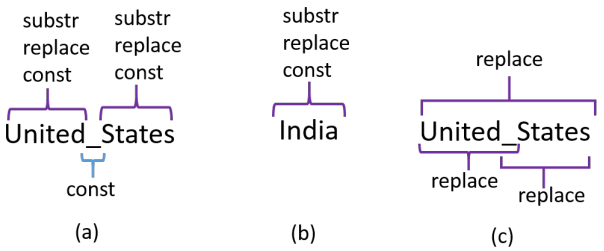


Fig. 10. (a) and (b) shows a portion of the DAGs for Ex 4.3 using layer 1 of hierarchical search. (c) shows additional edges from layer 2 of the hierarchical search.

Example 4.2. Now, for the same example, assume that we want to get the currency exchange values from http://finance.yahoo.com/q?s=EURUSD=X. For this example, layers 1 and 2 can only learn the string EURUSD as a ConstStr which will not work for the other inputs, or a AnyStr which is too general. So, the layered search moves to layer 3 which allows SubStr inside words. Now, the system can learn EUR and USD separately as SubStr expressions and concatenate them.

Example 4.3. Consider another example where we want to learn the URL https://en.wikipedia.org/wiki/United_States from the input United States and the URL https://en.wikipedia.org/wiki/India from the input India. Fig. 10(a) and (b) show portions of the DAGs for these two examples when using the first layer[1]. Here, there is no common program that can learn both these examples together. In such situations, the hierarchical search will move to the second layer. Fig. 10(c) shows the extra edges added in layer 2. Now, these examples can be learned together as Concat(ConstStr("https"),$\cdots$,ConstStr("/"), Replace(ConstPos(0),ConstPos(-1), " ", "_")).

4.2.3 Output-constrained Ranking. The LearnURL algorithm learns multiple programs that match the example URLs. However, not all of these programs are desirable for two reasons. First, some filter programs are too general and hence, cannot uniquely identify the desired URLs from the search results. For instance, AnyStr is one of the possible predicates that will be learned by the algorithm, but in addition to matching the example URLs, this predicate will also match any URL in the search results. Hence, we need to carefully select a consistent program from the DAG. Second, not all programs are equally desirable as they may not generalize well to the unseen inputs. For instance, consider Ex 2.2. If we have an example URL such as https://weather.com/weather/today/l/Seattle+WA+98109:4:US#!, then the programs that make the zip code 98109 to be a constant instead of AnyStr are not desirable. On the other hand, we want strings such as today, weather to be constants. We overcome both of these issues by devising an output-constrained ranking scheme.

[1]We omit the DAGs for the first part of the URLs as they are similar to the previous example.

```
GenProg({(v_i, o_i)}_i, λ_s, λ_c, λ_a)
 Dag d = GenDag(v_1, o_1, λ_s, λ_c, λ_a)
 for i from 2 to n:
    if d = null: return null
    Dag d' = GenDag(v_i, o_i, λ_s, λ_c, λ_a)
    d = d.Intersect(d')
 return SearchBestProg(d)
```

```
GenDag(v, o, λ_s, λ_c, λ_a)
 V = {0, ..., len(o)}, v_s = {0}, v_t = {len(o)}
 E = {< i,j > : 0 ≤ i < j ≤ len(o)}
 W = maps each edge to set of atomic exprs
 foreach 0 ≤ i < j ≤ len(o):
    w = ∅
    if λ_s(i,j,o): w.add(GenSubstr(i,j, v, o))
    if λ_s(i,j,o): w.add(GenReplace(i,j, v, o))
    if λ_c(i,j,o): w.add(ConstStr(v[i..j]))
    if λ_a(i,j,o): w.add(AnyStr)
    W.add(<i, j>,w)
 return Dag(V, v_s, v_t, E, W)
```

Fig. 11. Synthesis algorithm for URL generation programs, parameterized by λ_s, λ_c, and λ_a.

```
LearnURL({(v_i, o_i)}_i)
 if(p := GenProg({(v_i, o_i)}_i, oW, oW, oW) ≠ ⊥: return p // Layer 1
 if(p := GenProg({(v_i, o_i)}_i, mW, oW, oW) ≠ ⊥: return p // Layer 2
 if(p := GenProg({(v_i, o_i)}_i, iW ∨ mW, oW, oW) ≠ ⊥: return p // Layer 3
 return GenProg({(v_i, o_i)}_i, T, T, T) // Layer 4
```

Fig. 12. Layered version spaces for learning a program in L_u.

Fig. 13 shows our approach where we search for a consistent program in tandem with finding the best program. The algorithm takes as input a DAG d, a list of examples $\{v_i, o_i\}_i$, and a list of unseen inputs $\{v'_i\}_i$, and the outcome is the best program in the DAG that is consistent with the given examples (if such a program exists). The algorithm is a modification to Dijkstra's shortest path algorithm and uses a ranking scheme that ranks the atomic expressions in the language as `SubStr` = `Replace` > `ConstStr` > `AnyStr`. The algorithm maintains a ranked list of at-most k *prefix programs* for each node v in the DAG where a prefix program is a path in the DAG from the start node to v and the rank of a prefix program is the sum of ranks of all atomic expressions in the path. Initially, the set of prefix programs for every node is $\emptyset$. The algorithm then traverses the DAG in reverse topological order and for each outgoing edge, it iterates through pairs of prefixes and atomic expressions based on their ranks and checks if the pair is consistent with the given examples. Whenever a consistent pair is found, the concatenation of the atomic expression with the prefix is added to the list of prefixes for the other node in the edge. Finally, the algorithm returns the best prefix for the target node of the DAG. In the limit $k \rightarrow \infty$, the above algorithm will always find a consistent program if it exists. However, in practice, we found that a smaller value of k is sufficient because of the two pruning steps at Line 12 and Line 13 and because of our ranking that gives least preference to `AnyStr`.

For finding good generalizable programs, we prune away programs that do not result in valid URLs for the other unseen inputs by adding an additional check at Line 9 in Fig. 13. With this additional check, now, the constant `98109` will most probably not occur in the search results for other addresses such as `Cambridge` and hence, that atomic expression will not be considered in the best program.

Theorem 4.4 (Soundness of GenProg). *The* `GenProg` *algorithm is sound for all λ_s, λ_c and λ_a i.e. given a few input-output examples $\{(v_i, o_i)\}_i$, the learned program P will always satisfy $\forall i.\ [\![P]\!]_{v_i} = o_i$.*

```
1  SearchBestProg(d, {(v_i, o_i)}_i, {v'_i}_i)
2  foreach v ∈ V(d):
3     v.prefixes = ∅,
4  foreach v ∈ V(d) in reverse topological order:
5     foreach e: <v, v'> ∈ E(d):
6        foreach φ_sofar in v.prefixes.RankedIterator:
7           foreach f in W(e).RankedIterator:
8              if Consistent(φ_sofar + f, v', {(v_i, o_i)}_i):
9                 if Generalizes(φ_sofar + f, v', {v'_i}_i):
10                   v'.prefixes.add(φ_sofar + f,
11                    φ_sofar.score + f.score)
12                   if f ≠ AnyStr: Goto Line 13
13          if AnyStr ∉ φ_sofar: Goto Line 5
14 return d.v_t.prefixes[0]
```

Fig. 13. Algorithm to find the best consistent program from the DAG learned by LearnURL.

Proof: This holds because the GenDag algorithm only learns programs that are consistent for each input and Intersect preserves this consistency across multiple inputs. For learned programs that contain AnyStr expressions, the SearchBestProg algorithm ensures soundness because it only adds an atomic expression to a prefix if the combination is consistent with respect to the given examples.

Theorem 4.5 (Completeness of GenProg). *The* GenProg *algorithm is complete when* λ_s = *True,* λ_c = *True,* λ_a = *True, and in the limit* $k \to \infty$ *where at-most* k *prefixes are stored for each node in the* SearchBestProg *algorithm. In other words, if there exists a program in* L_u *that is consistent with the given set of input-output examples, then the algorithm will produce one.*

Proof: This is because when λ_s, λ_c, and λ_a are True, GenDag will learn all atomic expressions for every edge that satisfy the examples. Since, the DAG structure allows all possible concatenations of atomic expressions, the GenDag algorithm is complete in this case. Intersect also preserves completeness, and in the limit $k \to \infty$, the SearchBestProg will try all possible paths in the DAG to find a consistent program. Thus, GenProg does not drop any consistent program and hence, complete.

Theorem 4.6 (Soundness of LearnUrl). *The* LearnUrl *algorithm is sound.*

Proof: This is because every layer in the layered search is sound using Theorem 4.4.

Theorem 4.7 (Completeness of LearnUrl). *The* LearnUrl *algorithm is complete in the limit* $k \to \infty$ *where at-most* k *prefixes are stored for each node in the* SearchBestProg *algorithm.*

Proof: This follows because the last layer in the layered search is complete using Theorem 4.5.

5 DATA EXTRACTION LEARNING

Once we have a list of URLs, we now need to synthesize a program to extract the relevant data from these web pages. This data extraction can be done using a query language such as XPath [Berglund et al. 2003] that uses path expressions to select an HTML node (or a list of HTML nodes) from an HTML document. Previous systems such as DataXFormer [Abedjan et al. 2015; Morcos et al. 2015] have considered the absolute XPath obtained from the examples to do the extraction on other unseen inputs. An absolute XPath assumes that all the elements from the root of the HTML document to the target node have that same structure. This assumption is not always valid as

$$
\begin{array}{rcl}
\text{Prog } P & := & (\texttt{name}, \{\pi_1, \cdots, \pi_r\}) \\
\text{Pred } \pi & := & \pi_n \mid \pi_{\texttt{path}} \\
\text{NodePred } \pi_n & := & \pi_a \mid \pi_c \\
\text{AttrPred } \pi_a & := & [\texttt{attr(name)} == \texttt{e}] \\
\text{CountPred } \pi_c & := & [\texttt{count(axis)} == k] \\
\text{PathPred } \pi_{\texttt{path}} & := & [p] \\
\text{Path } p & := & n_c \mid n_s \mid n_a/n_s \mid p/n_c \\
\text{Node } n & := & (\texttt{name}, \texttt{axis}, \pi_{\texttt{pos}}, \{\pi_{n_1}, \cdots, \pi_{n_r}\}) \\
\text{PosPred } \pi_{\texttt{pos}} & := & [\texttt{pos } (== \mid \leq)\ k] \mid \bot \\
\text{Axis axis} & := & \texttt{Child} \mid \texttt{Ancestor} \mid \texttt{Left} \mid \texttt{Right} \\
\text{String e} & := & \texttt{Same as e in Fig. 6}
\end{array}
$$

Fig. 14. The syntax for the extraction language L_w, where name is a string, k is an integer, and $\bot$ is an empty predicate. (b) Semantics of L_w, where AllNodes, Len and Neighbors are macros with expected semantics.

$$
\begin{array}{rcl}
[\![(\texttt{name}, \{\pi_1, \cdots, \pi_r\})]\!]_v(d) & = & \texttt{Filter}(\lambda\gamma.\gamma.\texttt{name} == \texttt{name and } \wedge_{i=1}^{r} [\![\pi_i]\!]_v(\gamma), \texttt{AllNodes}(d)) \\
[\![[\texttt{attr(name)} == e]]\!]_v(\gamma) & = & \gamma.\texttt{Attr(name)} == [\![e]\!]_v \\
[\![[\texttt{count(axis)} == k]]\!]_v(\gamma) & = & \texttt{Len}(\texttt{Neighbors}(\gamma, \texttt{axis})) == k \\
[\![[p]]\!]_v(\gamma) & = & \texttt{Len}([\![p]\!]_v(\{\gamma\})) > 0 \\
[\![p/n]\!]_v(\{\gamma_i\}_i) & = & [\![n]\!]_v([\![p]\!]_v(\{\gamma_i\}_i)) \\
[\![n]\!]_v(\{\gamma_i\}_i) & = & \texttt{Filter}(\lambda\gamma.\texttt{Check}_v(n, \gamma), \\
 & & \quad \texttt{Flatten}(\texttt{Map}(\lambda\gamma.\texttt{Neighbors}(\gamma, n.\texttt{axis}, n.\pi_{\texttt{pos}})), \{\gamma_i\}_i))) \\
[\![e]\!]_v & = & \texttt{Same as in Fig. 7} \\
\texttt{Check}_v(n, \gamma) & \equiv & \gamma.\texttt{name} == n.\texttt{name and } \wedge_i [\![n.\pi_{n_i})]\!]_v(\gamma)
\end{array}
$$

Fig. 15. The semantics of L_w, where AllNodes, Len and Neighbors are macros with expected semantics.

web-pages are very dynamic. For instance, consider the weather data extraction from Ex 2.2. The web pages sometimes have an alert message to indicate events such as storms, and an absolute XPath will fail to extract the weather information from these web pages. More importantly, an absolute XPath will not work for input-dependent data extractions as shown in Ex 1.1.

Learning an XPath program from a set of examples is called *wrapper induction*, and there exist many techniques [Anton 2005; Dalvi et al. 2009; Kushmerick 1997; Muslea et al. 1998] that solve this problem. However, none of the previous approaches applied wrapper induction in the context of data-integration tasks that requires generating input-dependent XPath programs. In this section, we first present a DSL that extends the XPath language with input-dependent constructs, and then a sound and complete synthesis algorithm that can learn robust and generalizable extraction programs in this DSL using very few examples.

5.1 Data Extraction Language L_w

Syntax. Fig. 14 shows the syntax of L_w. At the top-level, a program is a tuple containing a name and a list of predicates, where name denotes the HTML tag and predicates denote the constraints that the desired "target" nodes should satisfy. There are two kinds of predicates—NodePred and

PathPred. A NodePred can either be an AttrPred or a CountPred. An AttrPred has a name and a value. We treat the text inside a node as yet another attribute. The attribute values are not just strings but are string expressions from the DSL L_u in Fig. 6, which allows the attributes to be computed using string transformations on the input data. This is one of the key differences between the XPath language and L_w. A CountPred indicates the number of neighbors of a node along a particular direction. Predicates can also be PathPreds denoting existence of a particular path in the HTML document starting from the current node. A path is a sequence of nodes where each node has a name, an axis, a PosPred, and a list of NodePreds (possibly empty). The name denotes the HTML tag, axis is the direction of this node from the previous node in the path, and PosPred denotes the distance between the node and the previous node along the axis. A PosPred can also be empty ($\perp$) meaning that the node can be at any distance along the axis. In L_w, we only consider paths that have at-most one node along the Ancestor axis (n_a) and at-most one sibling node along the Left or the Right axis (n_s). Moreover, the ancestor and sibling nodes can only occur at the beginning of the path.

Semantics. Fig. 15 shows the semantics of L_w. A program P is evaluated under a given input data v on an HTML document d, and it produces a list of HTML nodes that have the same name and satisfy all the predicates in P. In this formulation, we use γ to represent an HTML node, and it is not to be confused with the node n in the DSL. The predicates are evaluated on an HTML node and results in a Boolean value. Evaluating an AttrPred checks whether the value of the attribute in the HTML node matches the evaluation of the string expression under the given input v. A CountPred verifies that the number of children of the HTML node along the axis (obtained using the Neighbors macro) matches the count k. A PathPred first evaluates the path which results in a list of HTML nodes and checks that the list is not empty. A path is evaluated step-by-step for each node where each step evaluation is based on the set of HTML nodes obtained from the previous steps. Based on the axis of the node in the current step evaluation, the set of HTML nodes is expanded to include all their neighbors along that axis and at a position as specified by the PosPred. Next, this set is filtered according to the name of the node and its node predicates (using the Check macro).

Example. A possible program for the currency exchange rate extraction in Ex 1.1 is `(td, [(td, Left,[pos == 1])/ (text,Child,[attr("text") ==` ⟨**Transformed Date**⟩`])])`. This expression denotes the extraction of an HTML node (γ_1) with a td tag. The path predicate states that there should be another td HTML node (γ_2) to the left of γ_1 at a distance of 1 and it should have a text child with its text attribute equal to the transformed date that is learned from the input.

Design choices. This DSL is only a subset of the XPath language that has been chosen so that it can handle a wide variety of data extraction tasks and at the same time enables efficient synthesis algorithm. For example, our top-level program is a single node whereas the XPath language would support arbitrary path expressions. Moreover, paths in XPath do not have to satisfy the ordering criteria that L_w enforces and in addition, the individual nodes in the path expression in L_w cannot have recursive path predicates. However, we found that most of these constraints can be expressed as additional path predicates in the top-level program and hence, does not restrict the expressiveness.

5.2 Synthesis Algorithm

We now describe the synthesis algorithm for learning a program in L_w from examples. Here, the list of input-output examples is denoted as $E = \{(i_1, w_1, \gamma_1), (i_2, w_2, \gamma_2), \cdots (i_m, w_m, \gamma_m)\}$, where each example is a tuple of an input i, a web page w, and a target HTML node γ, and the list of pairs of unseen inputs and web pages is denoted as $U = \{(i_{m+1}, w_{m+1}), (i_{m+2}, w_{m+2}), \cdots, (i_{m+n}, w_{m+n})\}$.

$$
\begin{aligned}
\texttt{LearnTarget}(\gamma) &= \texttt{LearnAnchor}(\gamma); \texttt{LearnChildren}(\gamma); \texttt{LearnSiblings}(\gamma); \texttt{LearnAncestors}(\gamma) \\
\texttt{LearnChildren}(\gamma) &= \forall \gamma' \texttt{ in Neighbors}(\gamma, \texttt{Child}).\ \texttt{LearnAnchor}(\gamma'); \texttt{LearnChildren}(\gamma') \\
\texttt{LearnSiblings}(\gamma) &= \forall \gamma' \texttt{ in Neighbors}(\gamma, \{\texttt{Left}, \texttt{Right}\}).\ \texttt{LearnAnchor}(\gamma'); \texttt{LearnChildren}(\gamma') \\
\texttt{LearnAncestors}(\gamma) &= \forall \gamma' \texttt{ in Neighbors}(\gamma, \texttt{Ancestor}).\ \texttt{LearnAnchor}(\gamma'); \texttt{LearnSiblings}(\gamma') \\
\texttt{LearnAnchor}(\gamma) &= \texttt{Anchor}(\gamma.\texttt{name}, \texttt{LearnPreds}(\gamma)) \\
\texttt{LearnAttrPred}(\gamma) &= \forall \texttt{attr in } \gamma.\ \texttt{AttrPred}(\texttt{attr.name}, \texttt{GenDag}(\texttt{attr.value})) \\
\texttt{LearnCountPred}(\gamma) &= \forall \texttt{dir}.\ \texttt{CountPred}(\texttt{dir}, \texttt{Len}(\texttt{Neighbors}(\gamma, \texttt{dir})))
\end{aligned}
$$

Fig. 16. Algorithm to transform an HTML document into a predicates graph.

In this case, the O-PBE synthesis task can be framed as a search problem to find the right set of predicates ($\{\pi_1, \pi_2, \cdots, \pi_r\}$) that can sufficiently constrain the given target nodes.

At a high-level, the synthesis algorithm has three key steps: First, it uses the first example to learn all possible predicates for the target node in that example. Then, the remaining examples are used to refine these predicates. Finally, the algorithm searches for a subset of these predicates that satisfies the given examples and also generalizes well to the unseen inputs.

5.2.1 Learning all predicates. For any given example HTML node, there are numerous predicates (especially path predicates) in the language that constrains this target node. In order to learn and operate on these predicates efficiently, we use a graph data structure called *predicates graph* to compactly represent the set of all predicates. This data structure is inspired by the tree data structure used in Anton [2005], but it is adapted to our DSL and algorithm.

To avoid confusion with the nodes in the DSL, we use the term `Anchor` to refer to nodes and the term `Hop` to refer to edges in this graph. Hence, a predicates graph is a tuple (A, H, T) where A is the list of anchors, H is the list of hops and $T \in A$ is the target anchor. An anchor is a tuple (n, P) where n is the name of the anchor and P is a list of node predicates in the L_w language. An edge is a tuple (a_1, a_2, x, d) where a_1 is the start anchor, a_2 is the end anchor, x is the axis and d is a predicate on the distance between a_1 and a_2 measured in terms of number of hops in the original HTML document.

Fig. 16 shows the algorithm for transforming an HTML document into a predicates graph. We will explain this algorithm based on an example shown in Fig. 17 where the input HTML document is shown on the left, and the corresponding predicates graph is shown on the right; the target node is the text node (T_3) shown in red. First, it is important to note the difference between these two representations. Although both the HTML document and the predicates graph have a tree like structure, the latter is more centered around the target anchor. In the predicates graph, all anchors are connected to the target anchor using a minimum number of intermediate anchors that is allowed by the DSL. The algorithm first creates an anchor for the target and then learns the anchors of its children, siblings, and ancestors recursively. Learning a child or a sibling will also learn its children in a recursive manner, whereas learning an ancestor will only learn its siblings recursively. Finally, when creating an anchor for an HTML node, all the node predicates of the HTML node are inherited by the anchor, but if there are any attribute predicates, their string values are first converted to DAGs using the `GenDag` method in § 4.2.2.

After the above transformation, a path $p = a_1/a_2/\cdots a_r$ in the predicates graph (where a_1 is the target node) represents many different predicates in L_w corresponding to different combinations of the node predicates in each anchor a_i, e.g. the path from the target (T_3) to the text node T_2 in Fig. 17 can be translated to the following path predicates:

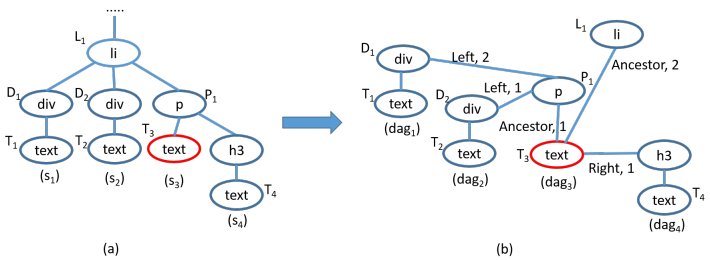


Fig. 17. An example demonstrating the transformation from a HTML document to a predicates graph.

π_{p_1} = `[(P, Ancestor, [Pos == 1])/(div, Left, [Pos == 1])/ (text, Child, [attr["text"] = dag2.TopProg])]`
π_{p_2} = `[(P, Ancestor, [Pos == 1])/(div, Left)/(text, Child, [attr["text"] = dag2.TopProg])]`
π_{p_3} = `[(P, Ancestor)/(div, Left)/(text, Child)]`

We can define a partial ordering among predicates generated by path p as follows: $\pi_1 \sqsubseteq \pi_2$ if the set of all node predicates in π_1 is a subset of the set of all node predicates in π_2. In the above example, we have $\pi_{p_3} \sqsubseteq \pi_{p_2} \sqsubseteq \pi_{p_1}$.

Definition 5.1 (Minimal path predicate). Given a path p in the predicates graph, a minimal path predicate is the predicate π_p encoded by this path such that $\nexists \pi_{p'}.\ \pi_{p'} \sqsubseteq \pi_p$.

Definition 5.2 (Maximal path predicate). Given a path p in the predicates graph, a maximal path predicate is the predicate π_p such that $\nexists \pi_{p'}.\ \pi_p \sqsubseteq \pi_{p'}$

In other words, a *minimal path predicate* is the one that does not have any node predicates for any anchor in the path and a *maximal path predicate* is the one that has all node predicates for every anchor in the path. For the above example, π_{p_3} is the *minimal path predicate* and π_{p_1} is the *maximal path predicate* assuming the nodes do not have any other predicates.

Lemma 5.3. *The predicates graph for an example* (i_k, w_k, γ_k) *can express all predicates in* L_w *that satisfy the example.*

Proof Sketch: This lemma is true because the traversal of nodes when constructing the predicates graph covers all possible paths in the HTML document that can be expressed in L_w.

Lemma 5.4. *Any predicate expressed by the predicates graph for an example* (i_k, w_k, γ_k) *will satisfy the example.*

Proof Sketch: This lemma is true because `LearnTarget` constructs predicates graph by only adding those nodes and predicates that are in the original HTML document.

For the implementation, we only construct a portion of the predicates graph that captures nodes that are within a distance $r = 5$ from the target node.

5.2.2 Handling multiple examples. We, now, have a list of all predicates that constrain the target HTML node for the first example. However, not all of these predicates will satisfy the other examples provided by the user. A simple strategy to prune the unsatisfiable predicates will be to create a predicates graph for each example and perform a full intersection of all these graphs. However, this operation is very expensive and has a complexity of N^m where N is the number of nodes in the HTML document and m is the number of examples. Moreover, in the next subsection, we will see that the algorithm for searching the right set of predicates (Fig. 20) will try to add these predicates

$$
\begin{aligned}
\texttt{IntersectPath}(p, q) &= \{a_i\}_i \texttt{ where } a_i = \texttt{IntersectAnchor}(a_{p_i}, a_{q_i}) \\
\texttt{IntersectAnchor}(a_p, a_q) &= \texttt{Anchor}(a_p.\texttt{name}, \{\pi_i\}_i) \texttt{ where } \pi_i = \texttt{IntersectPred}(\pi_{p_i}, \pi_{q_i}) \\
\texttt{IntersectAttrPred}(\pi_p, \pi_q) &= \texttt{AttrPred}(\pi_p.\texttt{name}, \texttt{Intersect}(\pi_p.\texttt{DAG}, \pi_q.\texttt{DAG})) \texttt{ if } \pi_p.\texttt{name} = \pi_q.\texttt{name} \\
\texttt{IntersectPosPred}(\pi_p, \pi_q) &= \texttt{PosPred}(==, \pi_p.k) \texttt{ if } \pi_p.k = \pi_q.k \texttt{ else PosPred}(\leq, \max(\pi_p.k, \pi_q.k)) \\
\texttt{IntersectCountPred}(\pi_p, \pi_q) &= \texttt{CountPred}(\pi_p.k) \texttt{ if } \pi_p.k = \pi_q.k
\end{aligned}
$$

Fig. 18. Algorithm to intersect two paths in two predicates graphs.

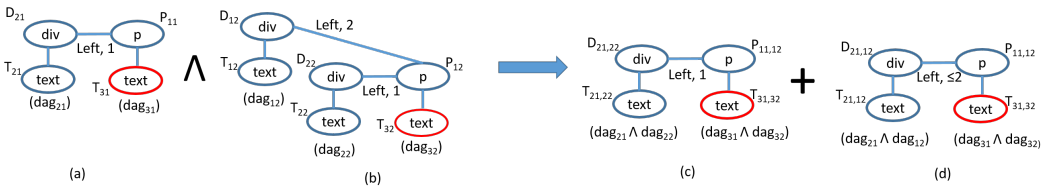


Fig. 19. Example demonstrating refining a path in predicates graph using other examples.

one-by-one based on a ranking scheme and stops after a satisfying set is obtained. Therefore, our strategy is to refine predicates in the predicates graph in a lazy fashion for one path at a time (rather than the whole graph) when the path is required by the Search algorithm.

We will motivate this refinement algorithm using the example from Fig. 17. Suppose that the first example E_1 in this scenario has i_1 = "10/16/16" and w_1 as shown in Fig. 17(a) with s_{1_1} = "10-16-2016" and s_{2_1} = "foo". As described earlier, one of the possible predicates for this example is:
π = `[(p,Ancestor,[pos==1])/(div,Left,[pos==1])/ (text,Child, [attr["text"]=dag2.TopProg])]`.
Now, suppose that the top program of $\texttt{dag}_2$ extracts the date (16) in the text of s_{1_1} from the year (16) (rather than the date 16) in the input i_1. Also, assume that the second example E_2 has i_2 = "10/15/16"and w_2 similar to w_1 but with s_{1_2} = "bar" and s_{2_2} = "10-15-2016". Clearly, the predicate π will not satisfy this new example and hence, we need to refine the path for π using the other examples. In this case, the path for π is $p = T_{3_1}/P_{1_1}/D_{2_1}/T_{2_1}$ as shown in Fig. 19(a). The algorithm Refine iteratively refines this path p for one example at a time. For any particular example E_i, the algorithm will first check if the *maximal path predicate* of p satisfies the example. If it does, then there is no need to refine this path. Otherwise, the algorithm gets all paths in the predicates graph corresponding to E_i that satisfies the *minimal path predicate* of p. For the example E_2, there are two such paths as shown in Fig. 19(b). The algorithm will then intersect p with each of these paths.

Fig. 18 shows the algorithms for intersecting two paths. The algorithm goes through all the anchors in the two paths and intersects each of their node predicates. For intersecting attribute predicates, the algorithm will intersect their respective DAGs corresponding to the values if their attributes have the same name. For intersecting position predicates, the algorithm will take the maximum value of k and update the operation accordingly. For intersecting count predicates, the algorithm will return the same predicate if the counts (k) are the same. For the example in Fig. 19, Fig. 19(c)-(d) are the two new resulting paths after intersection, whose predicates are:
π'_1 = `[(p,Ancestor,[pos==1])/(div,Left,[pos==1])/(text,Child, [attr["text"]=(dag2_1 ∧ dag2_2).TopProg])]`
π'_2 = `[(p,Ancestor,[pos==1])/(div,Left,[pos≤ 2])/(text,Child, [attr["text"]=(dag2_1 ∧ dag1_2).TopProg])]`

```
SearchBestProg(Π, E, U)
Π′ = ϕ; RankP(Π)
foreach π in Π.SortedIterator:
    P = Refine(Path(π), E)
    foreach π′ in P(π)
        done = TestNAdd(π, E, U)
        if (done): return (E[0].γ.Name, Π′)
```

Fig. 20. The algorithm for computing the best predicate set using output-constrained ranking.

The worst case complexity of this lazy refinement strategy is N^m, but in practice, it works well because the algorithm usually accesses few paths in the predicates graph because of the ranking scheme and moreover, the path intersection is only required if the original path cannot satisfy the new examples.

Lemma 5.5. *Any predicate obtained after refining a path satisfies all the examples.*

Proof Sketch: This holds because the `IntersectPath` only adds those node predicates to the path that can be satisfied by the examples.

Lemma 5.6. *Refining a path preserves all predicates that satisfy the examples.*

Proof Sketch: This holds because the algorithm for refining a path intersects the path with all similar paths (that satisfy the minimal path predicate) in the other examples and `IntersectPath` preserves all predicates that can be satisfied by the examples.

5.2.3 Output-constrained ranking of predicates. We now have a list of predicates that satisfy the first example and we have an algorithm to refine predicates based on the other examples provided by the user. However, only some of these predicates are desirable as some predicates might not generalize to unseen inputs and some others might not be required to constrain the target nodes. Fig. 20 shows the algorithm that uses *output-constrained ranking* to find a subset of the predicates that generalizes well. The algorithm takes as inputs a set of predicates Π, a list of input-output examples E, and a list of unseen inputs U. It uses the unseen inputs as a test set to prune away predicates that do not generalize well. The algorithm iterates through the list of all predicates based on a ranking scheme and adds the predicate if there is at-least one node in each test document that satisfies the predicates added so far. This process is stopped if we find a set of predicates that uniquely constrains the target nodes in the provided examples.

Ranking Scheme. We use the following criterion that gives higher priority to predicates that best constrain the set of target nodes and at the same time, capture user intentions: (i) Attribute predicates with more sub-string expressions on input data are preferred over other attribute predicates, (ii) Left siblings nodes are preferred over right siblings because usually websites contain descriptor information to the left of values and in most cases, these descriptor information tend to be the same across websites, and (iii) Nodes closer to target are preferred over farther nodes.

Theorem 5.7 (Soundness). *The data extraction synthesis algorithm is sound i.e. given some input-output examples $\{(i_k, w_k, \gamma_k)\}_k$, the program P that is learned by the algorithm will always satisfy $\forall k.\ [\![P]\!]_{i_k}(w_k) = \{\gamma_k\}$.*

Proof Sketch: This theorem follows from Lemmas 5.4 and 5.5.

Theorem 5.8 (Completeness). *The data extraction synthesis algorithm is complete i.e. if there exists a program in L_w that satisfies the given I/O examples, then the algorithm is guaranteed to find one program that satisfies the examples.*

Proof Sketch: This theorem follows from Lemmas 5.3 and 5.6.

Note, that the `SearchBestProg` algorithm cannot influence the soundness or completeness argument because the set of all predicates obtained after refining every path in the predicates graph is a sound and complete solution. `SearchBestProg` only influences how well it generalizes to unseen inputs.

A Note on adding new rows and changing data sources. When adding new rows to the spreadsheet after the integration task has been performed, there might be concerns about whether the joined data for the old rows will be obsolete. For example, in Ex 2.1 or Ex 2.2, the stock values or the weather information would change presumable every second. However, the user still does not need to re-provide updated examples. This is because WebRelate learns programs in the DSLs and it can just re-execute the learned program directly and compute results for the new rows. Only in cases if the website DOM structure changes (which does not happen too often for major websites), the user would need to re-update the previously provided examples.

6 EVALUATION

In this section, we evaluate WebRelate on a variety of real-world web integration tasks. In particular, we evaluate whether the DSLs for URL learning and data extraction are expressive enough to encode these tasks, the efficiency of the synthesis algorithms, the number of examples needed to learn the programs, and the impact of layered version spaces and output-constrained ranking.

Benchmarks. We collected 88 web-integration tasks taken from online help forums and other anonymous sources. These tasks cover a wide range of domains including finance, sports, weather, travel, academics, geography, and entertainment. Each benchmark has 5 to 32 rows of input data in the spreadsheet. These 88 tasks decompose into 62 URL learning tasks and 88 extraction tasks. The set of benchmarks with unique URLs is shown in Fig. 21.

Experimental Setup. We implemented WebRelate in C#. All experiments were done using a dual-core Intel i5 2.40 GHz CPU with 8GB RAM. For each component, we incrementally provided examples until the system learned a program that satisfied all other inputs and we report the results of the final iteration.

6.1 URL learning

For each URL learning task, we run the layered search using 4 different configurations for the layers: 1. `L1 to L4`, 2. `L2 to L4`, 3. `L3 to L4`, and 4. `Only L4` where `L1`, `L2`, `L3`, and `L4` are as defined in Fig. 12. The last configuration essentially compares against the synthesis algorithm used in FlashFill (except for the new constructs).

Fig. 22(a) shows the synthesis times[2] required for learning URLs. We categorize the benchmarks based on the layer that has the desired program. We can see that for the `L1 to L4` configuration, WebRelate solves all the tasks in less than 1 second. The `L2 to L4` configuration performs equally well whereas the performance of `only L4` configuration is much worse. Only 28 benchmarks complete when given a timeout of 2 minutes.

We also perform an experiment to evaluate the impact of *output-constrained ranking* using the `L1 to L4` configuration for the layered search and report the results in Fig. 22(b). With output-constrained ranking, 85% of the benchmarks require only 1 example whereas without it, only 29% of benchmarks can be synthesized using a single example.

[2]excluding the time take to run search queries on the search engine

#	Description	#R	Example data item(s)	Website Domain
1-2	ATP players to ages/countries	5	(age 29) | Serbia and Montenegro	wikipedia
3-4	ATP players to latest tweet/total tweets	5	Long text | 2,324	twitter
5-7	ATP players to single/ranking/W-L	20	7 | ATP Rank #1 | 742-152	espn
8	ATP players to number of singles in 2016	7	7	espn.go
9	ATP players to rankings	5	1	atpworldtour
10	ATP players to rankings	5	#1	tennis
11	ATP players to rankings	5	66 (7th in the Open Era)	wikipedia
12	Addresses to population	7	668,342 (100% urban, 0% rural).	city-data
13-14	Addresses to population/zipcode	7	786,130 | 98101 Zip Code	zipcode
15-16	Addresses to weather/weather on date	7	51° | 51°	accuweather
17	Addresses to weather	7	57 ĂĆÂřF	timeanddate
18	Addresses to weather	7	52	weather
19-23	Addresses to weather stats	6	49|Partly sunny|74°|62|68°	accuweather
24	Airport code to airport name	5	Boston Logan International Airport (BOS)	virginamerica
25-26	Airport code to delay/name	5	Very few delays | Long text	flightstats
27	Airport code to terminal information	5	Long text	aa
28	Albums to genre	5	Rock	wikipedia
29-31	Authors to paper citations, max citation, title	5	Cited by 175 | Cited by 427 | Long text	scholar.google
32-35	Authors to different data from DBLP	5	Long Text|Alvin Cheung (12)|PLDI (9)|POPL (2)	dblp
36	Cities to population	6	21,357,000	worldpopulation
37	Company symbols to 1 year target prices	6	65	nasdaq
38	Company symbols to stock prices	6	59.87	finance.yahoo
39	Company symbols to stock prices	6	59.87	money.cnn
40	Company symbols to stock prices	6	59.87	quotes.wsj
41	Company symbols to stock prices	6	59.87	google
42	Company symbols to stock prices	6	59.00	marketwatch
43	Company symbols to stock prices	6	59.95	nasdaq
44	Company symbols to stock prices	6	$59.87	google
45	Company symbols to stock prices on date	6	60.61	google
46	Company symbols to stock prices on date	6	60.61	yahoo
47	Country names to population	6	1,326,801,576	worldometers
48	Country names to population	6	1,336,286,256	wikipedia
49	Cricket results for teams on different dates	5	Australia won by 3 wickets	espncricinfo
50-52	Cricket stats for two different teams	5	90 | 24 | 31.67	stats.espncricinfo
53	Currency exchange values	8	66.7870	finance.yahoo
54	Currency exchange values	8	INR/USD = 66.84699	themoneyconverter
55	Currency exchange values	8	66.7800	bloomberg
56	Currency exchange values	8	66.8043 INR	investing
57	Currency exchange values	8	66.779	xe
58	Currency exchange values	8	66.7914	exchange-rates
59	Currency exchange values based on date	8	64.1643 INR	investing
60	Currency exchange values based on date	8	66.778	exchange-rates
61	Currency exchange values based on date	8	64.2308481698	investing
62	Flight distance between two cities	5	2,487 Miles	cheapoair
63-65	Flight fares between two cities/cheapest/airline	5	$217 | Wednesday | $237	farecompare
66	Flight fares between two cities	5	$257	cheapflights
67	Flight travel time between two cities	5	6 hrs 11 mins	travelmath
68	Flight travel time between two cities	5	5 hours, 38 minutes	cheapflights
69	NFL teams to rankings	5	4-5, 3rd in AFC East	espn
70	NFL teams to rankings	5	26th	cbssports
71	NFL teams to rankings	5	26.7	nfl
72	NFL teams to rankings	5	#5	teamrankings
73	NFL teams to stadium names	32	New Era Field	espn
74	Nobel prize for different subjects based on year	5	Richard F. Heck	wikipedia
75	Nobel prize for different years based on subject	5	Long Text.	nobelprize
76	Nobel prize for different years based on subject	5	Richard F. Heck, Ei-ichi Negishi and Akira Suzuki	nobelprize
77-78	Novels to authors/genre	7	Charles Dickens | Historical novel	wikipedia
79	Novels to authors	8	Charles Dickens	goodreads
80	# of coauthored papers between two authors	10	Rastislav BodĂĆÂŋk (6)	dblp
81	Number of daily flights between two cities	5	26	cheapoair
82	People to profession	5	Long Text	zoominfo
83-84	Real estate properties to sale prices and stats	5	$1,452 | 1,804 sqft	zillow
85	Stock prices with names from same webpage	6	0.00	marketwatch
86-88	Video names to youtube stats	5	11,473,055 | 2,672,818,718 views | Jul 15, 2012	youtube

Fig. 21. The set of web data integration benchmarks with a brief description along with number of input rows (#R), the example data item(s), and the website domain from which the data is extracted.

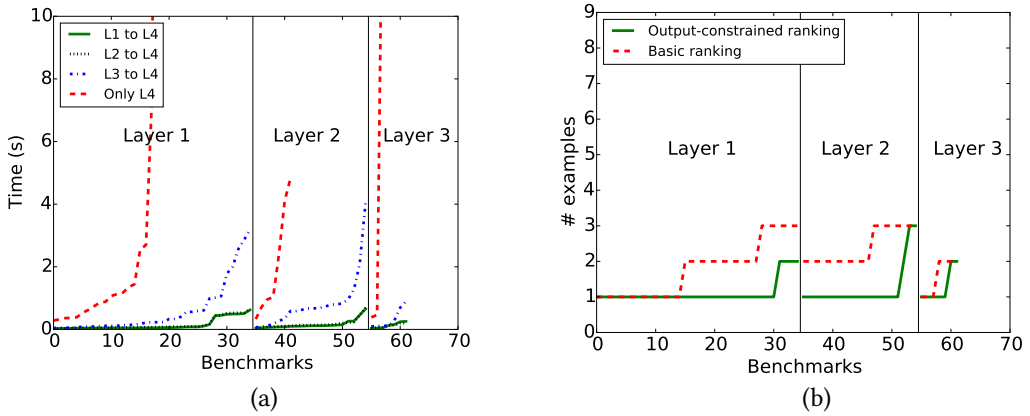


Fig. 22. (a) The synthesis times and (b) the number of examples required for learning URL programs.

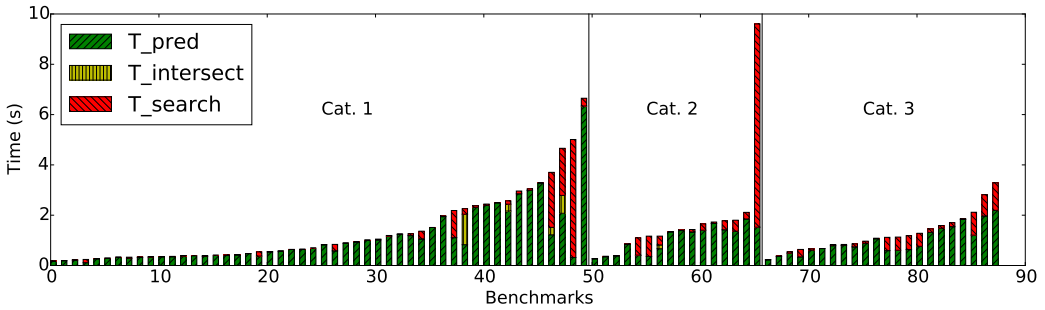


Fig. 23. The synthesis times for learning data extraction programs by WebRelate on the 88 benchmarks.

For these tasks, the length of the URLs is in the range of 23 to 89. Regarding DSL coverage, about 50% of the benchmarks require `AnyStr`, 88% require `SubStr`, 38% require `ReplaceStr`, and all benchmarks require `ConstStr` expressions. Thus, all DSL features are needed for different subsets of benchmarks.

To evaluate the scalability with respect to the number of examples, we performed an experiment where we took a benchmark that has 32 rows in the spreadsheet and incrementally added more examples. The results are shown in Fig. 24(a). Although the theoretical complexity of the algorithms is exponential in the number of examples, in practice, we observed that the performance scales almost linearly. We attribute this to the sparseness of the DAGs learned during the layered search.

6.2 Data Extraction

We now present the evaluation of our data extraction synthesizer. We categorize the benchmarks into three categories. The first category consists of benchmarks where the data extraction can be learned using an absolute XPath. The second category includes the benchmarks that cannot be learned using an absolute XPath, but by using relative predicates (especially involving nearby strings). The last category handles the cases where the data extraction is input-dependent.

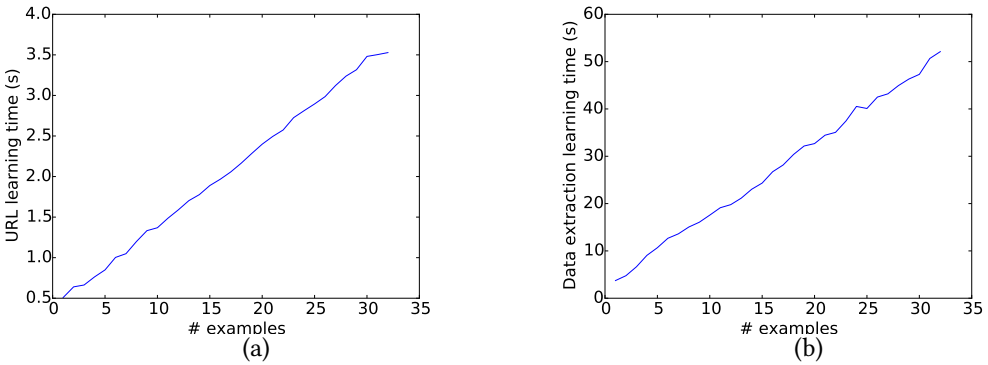


Fig. 24. (a) The synthesis time vs the number of examples for URL learning. (b) The synthesis time vs the number of examples for data extraction learning.

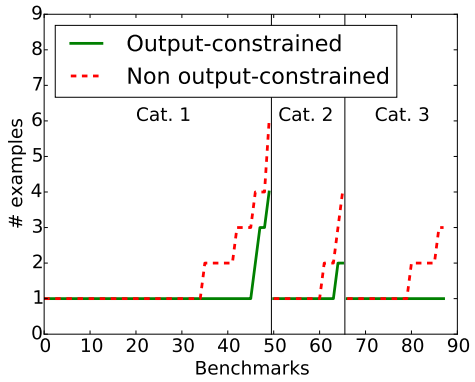


Fig. 25. The number of examples need to learn the data extraction programs.

Fig. 23 shows the synthesis times[3] for all 88 benchmarks. The figure also splits the synthesis time into the time taken for learning the predicates graphs (`T_pred`), for intersecting the predicates graphs if there are multiple examples (`T_intersect`) and finally, for searching for the right set of predicates (`T_search`). WebRelate can learn the correct extraction for all benchmarks in all three categories showing that the DSL is very expressive. Moreover, it can learn them very quickly—97% of benchmarks take less than 5 seconds. Fig. 25 shows the number of examples required and also compares against non output-constrained ranking. It can be seen that with output-constrained ranking, 93% of benchmarks require only 1 example.

Most of the synthesis time is actually spent in generating the predicates graphs and this time is proportional to the number of attribute strings in the HTML document since WebRelate will create a DAG for each string. In our examples, the number of HTML nodes we consider for the predicates graph is in the range of 30 to 1200 with 3 to 200 attribute strings. For some benchmarks, the time spent in searching for the right set of predicates dominates. These are the benchmarks that have require too many predicates to sufficiently constrain the target nodes. The actual time spent on intersecting the predicates graph is not very significant. For benchmarks that require

[3]excluding the time taken to load the webpages

multiple examples, we found that the time spent on intersection is only 15% of the total time (on average) due to our lazy intersection algorithm.

Fig. 24(b) shows how the performance scales with respect to the number of examples. Similar to URL learning, the performance scales almost linearly as opposed to exponentially. We attribute this to our lazy intersection algorithm.

7 RELATED WORK

Data Integration from Web: DataXFormer [Abedjan et al. 2015; Morcos et al. 2015] performs semantic data transformations using web tables and forms. Given a set of input-output examples, it searches a large index of web tables and web forms to find a consistent transformation. For web forms, it performs a web search consisting of input-output examples to identify web forms that might be relevant, and then uses heuristics to invoke the form with the correct set of inputs. Instead of being completely automated, WebRelate, on the other hand, allows users to identify the relevant websites and point to the desired data on the website, which allows WebRelate to perform data integration from more sophisticated websites. Moreover, WebRelate allows for joining data based on syntactic transformations on inputs, whereas DataXFormer only searches based on exact inputs.

WebCombine [Chasins et al. 2015] is a PBD web scraping tool for end-users that allows them to first extract logical tables from a webpage, and then provide example interactions for the first table entry on another website. It uses Ringer [Barman et al. 2016] as the backend record and replay engine to record the interactions performed by the user on a webpage. The recorded interaction is turned into a script that can be replayed programmatically. WebCombine parameterizes the recorded script using 3 rules that parameterize xpaths, strings, and frames with list items. Vegemite [Lin et al. 2009] uses direct manipulation and PBD to allow end-users to easily create web mashups to collect information from multiple websites. It first uses a demonstration to open a website, copy/paste the data for the first row into the website, and records user interactions using the CoScripter [Leshed et al. 2008] engine. The learnt script is then executed for remaining rows in the spreadsheet. The XPath learning does not need to be complex since all rows go to the same website and the desired data is at the same location in the DOM.

Instead of recording user demonstrations, WebRelate uses examples of URLs (or search queries) to learn a program to get to the desired webpages. The demonstrations-based specification has been shown to be challenging for users [Lau 2009] and many websites do not expose such interactions. These systems learn simpler XPath expressions for data extraction, whereas WebRelate can learn input data-dependent Xpath expressions that are crucial for data integration tasks. Moreover, these systems assume the input data is in a consistent format that can be directly used for interactions, whereas WebRelate learns additional transformations on the input for both learning the URLs and data-dependent extractions.

Programming By Examples (PBE) for string transformations: Data Wrangler [Kandel et al. 2011] uses examples and predictive interaction [Heer et al. 2015] to create reusable data transformations such as map, joins, aggregation, and sorting. There have also been many recent PBE systems such as FlashFill [Gulwani 2011], BlinkFill [Singh 2016], and FlashExtract [Le and Gulwani 2014] that use VSA [Polozov and Gulwani 2015] for efficiently learning string transformations from examples. Unlike these systems that perform data transformation and extraction from a single document, WebRelate joins data between a spreadsheet and a collection of webpages. WebRelate builds on top of the substring constructs introduced by these systems to perform both URL learning and data extraction. Moreover, WebRelate uses layered version spaces and a output-constrained ranking technique to efficiently synthesize programs.

There is another PBE system that learns relational joins between two tables from examples [Singh and Gulwani 2012]. It uses a restricted set of relational algebra to allow using VSA for efficient

synthesis. Unlike learning joins between two relational sources, WebRelate learns joins between a relational data source (spreadsheet) and a semi-structured data source (multiple webpages).

Wrapper Induction: Wrapper induction [Kushmerick 1997] is a technique to automatically extract relational information from webpages using labeled examples. There exists a large body of research on wrapper induction with some techniques using input-output examples to learn wrappers [Anton 2005; Dalvi et al. 2009; Hsu and Dung 1998; Kushmerick 1997; Le and Gulwani 2014; Muslea et al. 1998] and others [Chasins et al. 2015; Crescenzi et al. 2001; Zhai and Liu 2005] perform unsupervised learning using pattern mining or similarity functions. Some of these techniques [Anton 2005; Dalvi et al. 2009; Le and Gulwani 2014] are based on Xpath similar to WebRelate whereas some techniques such as [Kushmerick 1997] treat webpages as strings and learn delimiter strings around the desired data from a fixed class of patterns. The main difference between any of these techniques and WebRelate is that the extraction language used by WebRelate is more expressive as it allows richer Xpath expressions that can depend on inputs.

Program Synthesis: The field of program synthesis has seen a renewed interest in recent years [Alur et al. 2013]. In addition to VSA based approaches, several other approaches including constraint-based [Solar-Lezama et al. 2006], enumerative [Udupa et al. 2013], and stochastic [Schkufza et al. 2013] have been developed to synthesize programs in different domains. Synthesis techniques using examples have also been developed for learning data structure manipulations [Singh and Solar-Lezama 2011], type-directed synthesis for recursive functions over algebraic datatypes [Frankle et al. 2016; Osera and Zdancewic 2015], transforming tree-structured data [Yaghmazadeh et al. 2016], and interactive parser synthesis [Leung et al. 2015]. These approaches are not suited for learning URLs and data extraction programs because the DSL operators such as regular expression based substrings and data-dependent xpath expressions are not readily expressible in these approaches and enumerative approaches do not scale because of large search space.

8 LIMITATIONS AND FUTURE WORK

There are certain situations under which WebRelate may not be able to learn a task. First, since all of our synthesis algorithms are sound, WebRelate cannot deal with noise. For example, if any input data in the spreadsheet is misspelled or has a semantically different format, then we cannot learn such string transformation programs. Our system can handle syntactic data transformations and partial semantic transformations, but not fully semantic transformations. As future work, we plan to use machine learning techniques to learn a noise model based on semantics and web tables [He et al. 2015] and to incorporate this model into the synthesis algorithm. In WebRelate, we assume that the webpages containing the desired data have URLs that can be programmatically learned. However, there are also situations that require complex interactions such as traversing through multiple pages before getting to the page that has the data. In future, we want to explore integrating the techniques in this paper with techniques from record and replay systems such as Ringer [Barman et al. 2016] to enable data-dependent replay.

9 CONCLUSION

We presented WebRelate, a PBE system for joining semi-structured web data with relational data. The key idea in WebRelate is to decompose the task into two sub-tasks: URL learning and data extraction learning. We frame the URL learning problem in terms of learning syntactic string transformations and filters, whereas we learn data extraction programs in a rich DSL that allows for data-dependent Xpath expressions. The key idea in the synthesis algorithms is to use layered version spaces and output-constrained ranking to efficiently learn the programs using very few input-output examples. We have evaluated WebRelate successfully on several real-world web data integration tasks.

REFERENCES

Ziawasch Abedjan, John Morcos, Michael N Gubanov, Ihab F Ilyas, Michael Stonebraker, Paolo Papotti, and Mourad Ouzzani. 2015. Dataxformer: Leveraging the Web for Semantic Transformations.. In *CIDR*.

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *FMCAD*. 1–8.

Tobias Anton. 2005. XPath-Wrapper Induction by generalizing tree traversal patterns. In *Lernen, Wissensentdeckung und Adaptivitt (LWA) 2005, GI Workshops, Saarbrcken*. 126–133.

Shaon Barman, Sarah Chasins, Rastislav Bodík, and Sumit Gulwani. 2016. Ringer: web automation by demonstration. In *OOPSLA*. 748–764.

Anders Berglund, Scott Boag, Don Chamberlin, Mary F Fernandez, Michael Kay, Jonathan Robie, and Jérôme Siméon. 2003. Xml path language (xpath). *World Wide Web Consortium (W3C)* (2003).

Sarah Chasins, Shaon Barman, Rastislav Bodík, and Sumit Gulwani. 2015. Browser Record and Replay as a Building Block for End-User Web Automation Tools. In *WWW*. 179–182.

Valter Crescenzi, Giansalvatore Mecca, and Paolo Merialdo. 2001. Roadrunner: Towards automatic data extraction from large web sites. In *VLDB*, Vol. 1. 109–118.

Nilesh Dalvi, Philip Bohannon, and Fei Sha. 2009. Robust Web Extraction: An Approach Based on a Probabilistic Tree-edit Model. In *SIGMOD*. 335–348.

Jonathan Frankle, Peter-Michael Osera, David Walker, and Steve Zdancewic. 2016. Example-directed synthesis: a type-theoretic interpretation. In *POPL*. 802–815.

Mike Gualtieri. 2009. Deputize end-user developers to deliver business agility and reduce costs. *Forrester Report for Application Development and Program Management Professionals* (2009).

Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *POPL*. 317–330.

Sumit Gulwani, William R. Harris, and Rishabh Singh. 2012. Spreadsheet data manipulation using examples. *Commun. ACM* 55, 8 (2012), 97–105.

Yeye He, Kris Ganjam, and Xu Chu. 2015. SEMA-JOIN: Joining Semantically-related Tables Using Big Table Corpora. *Proc. VLDB Endow.* 8, 12 (Aug. 2015), 1358–1369.

Jeffrey Heer, Joseph M Hellerstein, and Sean Kandel. 2015. Predictive Interaction for Data Transformation.. In *CIDR*.

Chun-Nan Hsu and Ming-Tzung Dung. 1998. Generating finite-state transducers for semi-structured data extraction from the web. *Information systems* 23, 8 (1998), 521–538.

Sean Kandel, Andreas Paepcke, Joseph M. Hellerstein, and Jeffrey Heer. 2011. Wrangler: interactive visual specification of data transformation scripts. In *CHI*. 3363–3372.

Nicholas Kushmerick. 1997. *Wrapper induction for information extraction.* Ph.D. Dissertation. Univ. of Washington.

Tessa Lau. 2009. Why Programming-By-Demonstration Systems Fail: Lessons Learned for Usable AI. *AI Magazine* 30, 4 (2009), 65–67.

Vu Le and Sumit Gulwani. 2014. FlashExtract: A Framework for Data Extraction by Examples. In *PLDI*. 542–553.

Gilly Leshed, Eben M Haber, Tara Matthews, and Tessa Lau. 2008. CoScripter: automating & sharing how-to knowledge in the enterprise. In *CHI*. 1719–1728.

Alan Leung, John Sarracino, and Sorin Lerner. 2015. Interactive parser synthesis by example. In *PLDI*. 565–574.

James Lin, Jeffrey Wong, Jeffrey Nichols, Allen Cypher, and Tessa A Lau. 2009. End-user programming of mashups with vegemite. In *IUI*. 97–106.

John Morcos, Ziawasch Abedjan, Ihab Francis Ilyas, Mourad Ouzzani, Paolo Papotti, and Michael Stonebraker. 2015. DataXFormer: An Interactive Data Transformation Tool. In *SIGMOD*.

Ion Muslea, Steve Minton, and Craig Knoblock. 1998. Stalker: Learning extraction rules for semistructured, web-based information sources. In *Workshop on AI and Information Integration*. AAAI, 74–81.

Peter-Michael Osera and Steve Zdancewic. 2015. Type-and-example-directed Program Synthesis. In *PLDI*. 619–630.

Oleksandr Polozov and Sumit Gulwani. 2015. FlashMeta: a framework for inductive program synthesis. In *OOPSLA*. 107–126.

Eric Schkufza, Rahul Sharma, and Alex Aiken. 2013. Stochastic superoptimization. In *ASPLOS*. 305–316.

Rishabh Singh. 2016. BlinkFill: Semi-supervised Programming By Example for Syntactic String Transformations. *PVLDB* 9, 10 (2016), 816–827.

Rishabh Singh and Sumit Gulwani. 2012. Learning Semantic String Transformations from Examples. *PVLDB* 5, 8 (2012), 740–751.

Rishabh Singh and Armando Solar-Lezama. 2011. Synthesizing data structure manipulations from storyboards. In *FSE*. 289–299.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial sketching for finite programs. In *ASPLOS*. 404–415.

Abhishek Udupa, Arun Raghavan, Jyotirmoy V. Deshmukh, Sela Mador-Haim, Milo M. K. Martin, and Rajeev Alur. 2013. TRANSIT: specifying protocols with concolic snippets. In *PLDI*. 287–296.

Navid Yaghmazadeh, Christian Klinger, Isil Dillig, and Swarat Chaudhuri. 2016. Synthesizing transformations on hierarchically structured data. In *PLDI*. 508–521.

Yanhong Zhai and Bing Liu. 2005. Web Data Extraction Based on Partial Tree Alignment. In *WWW*. 76–85.